



Université Gaston Berger de Saint-Louis

Master Ingénierie Informatique et TIC

DBI1 - CONCEPTION DES BASES DE DONNEES

Vidéo Tutorielle :

Chapitre 3 : Objets avancés 2 (partie 2)

Les Triggers

Agenda

- ☐ Les séquences
- ☒ Les triggers



Les Triggers



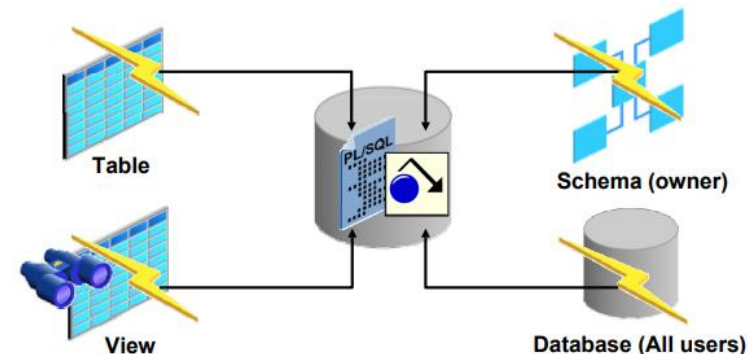
▪ Caractéristiques :

- Déclencheurs stockés dans la base et qui vont s'exécuter implicitement sur une action donnée.



▪ Actions ou Evènements

- Instructions manipulation de données, de structures, évènements utilisateurs (connexion..), systèmes (démarrage BD..)

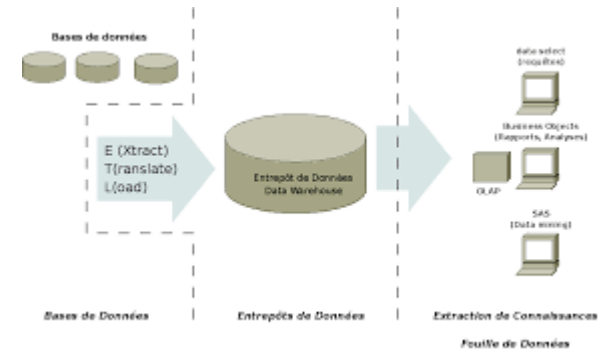


Les Triggers



■ Utilisation :

- Appliquer des règles métier qui ne peuvent pas être définies en utilisant des contraintes d'intégrité.
- Prendre le contrôle de la sécurité.
- Collecter des informations statistiques.
- Alimenter une base de données répliquée
- Générer automatiquement des valeurs pour les colonnes dérivées telles que la clé primaire numérique à incrémentation automatique.
- Empêcher une transaction invalide.



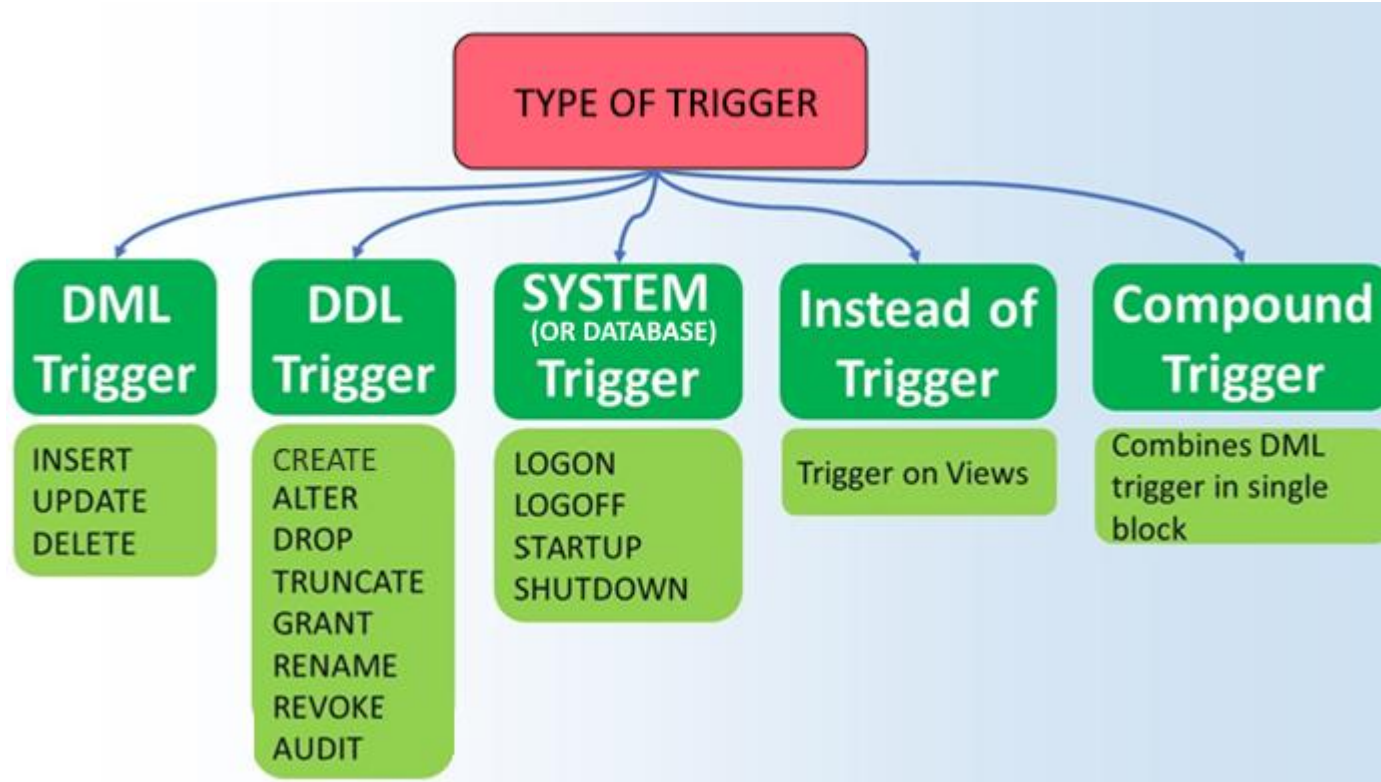
Les Triggers



▪ Restrictions :

- Taille maximale du corps du trigger ≤ 32 Ko
- Ne peut pas émettre d'instructions de contrôle de transaction ou d'instructions TCL telles que COMMIT, ROLLBACK ou SAVEPOINT.
- Ne peut utiliser des déclarations de variable de type LONG ou LONG RAW.
- Ne peut être appelé d'une manière récursive

Les Triggers : différents types



Les Triggers DML : table et Row



- **Quand :** BEFORE | AFTER
 - Pour trigger de niveau table : déclenché avant ou après l'évènement
 - Pour trigger de niveau ligne : déclenché avant ou après la modification de *chaque ligne* concernée (FOR EACH ROW)

- **Quoi :**
 - Corps Trigger : bloc PL/SQL effectué quand le trigger est déclenché
 - En plus :
 - IF INSERTING THEN ... END IF;
 - IF DELETING THEN ... END IF;
 - IF UPDATING THEN ... END IF;

Les Triggers DML



- **Attribut** : :OLD | :NEW
- Dans le corps du trigger (ou la clause WHERE) on peut se référer à la valeur d'un attribut avant ou après que soit effectuée la modification déclenchant le trigger :
 - :OLD.nomattribut : valeur avant la transaction UPDATE ou DELETE
 - :NEW.nomattribut : valeur après la transaction UPDATE ou INSERT

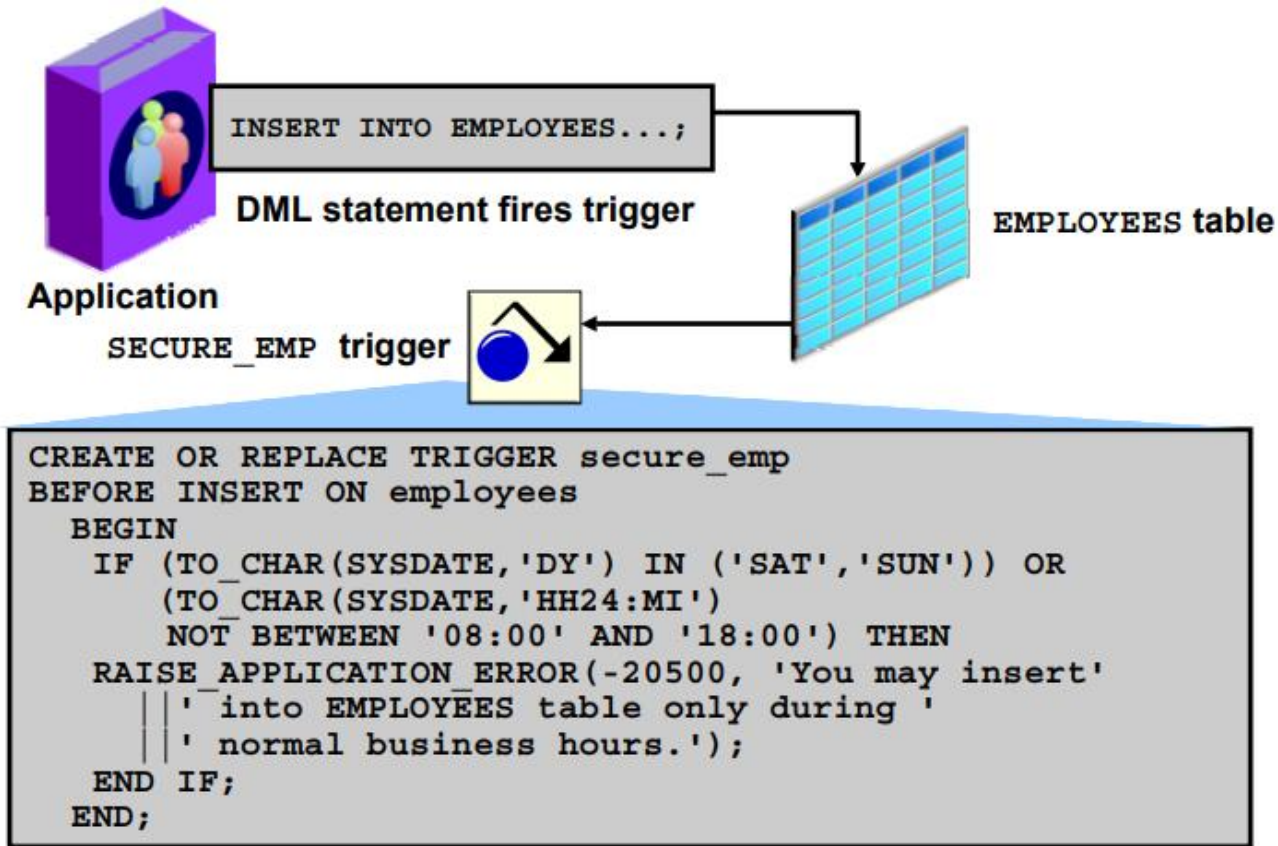
DML Pseudo Records	Insert	Update	Delete
New	✓	✓	✗
Old	✗	✓	✓

▪ Syntaxe :

```
CREATE [OR REPLACE] TRIGGER [schema.]nomtrigger  
    {BEFORE | AFTER | INSTEAD OF } listeEvenement  
    ON nomtable  
    [FOR EACH ROW]  
    [WHEN condition]  
    Trigger_Body  
END;
```



▪ Exemple 1: Trigger niveau table



▪ Exemple 1: Trigger niveau table (suite)

```
INSERT INTO employees (employee_id, last_name,  
    first_name, email, hire_date,  
    job_id, salary, department_id)  
VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE,  
    'IT_PROG', 4500, 60);
```

```
INSERT INTO employees (employee_id, last_name, first_name, email, hire_date,  
VALUES (300, 'Smith', 'Rob', 'RSMITH', SYSDATE, 'IT_PROG', 4500, 60)
```

Error report:

SQL Error: ORA-20500: You may insert into EMPLOYEES table only during business hours.

ORA-06512: at "ORA42.SECURE_EMP", line 4

ORA-04088: error during execution of trigger 'ORA42.SECURE_EMP'

▪ Exemple 1: Trigger niveau table (suite)

```
CREATE OR REPLACE TRIGGER secure_emp BEFORE
INSERT OR UPDATE OR DELETE ON employees
BEGIN
    IF (TO_CHAR(SYSDATE,'DY') IN ('SAT','SUN')) OR
       (TO_CHAR(SYSDATE,'HH24')
        NOT BETWEEN '08' AND '18') THEN
        IF DELETING THEN RAISE_APPLICATION_ERROR(
            -20502,'You may delete from EMPLOYEES table' ||
            'only during normal business hours.');
```

```
        ELIF INSERTING THEN RAISE_APPLICATION_ERROR(
            -20500,'You may insert into EMPLOYEES table' ||
            'only during normal business hours.');
```

```
        ELIF UPDATING ('SALARY') THEN
            RAISE_APPLICATION_ERROR(-20503, 'You may ' ||
            'update SALARY only normal during business hours.');
```

```
        ELSE RAISE_APPLICATION_ERROR(-20504,'You may' ||
            ' update EMPLOYEES table only during' ||
            ' normal business hours.');
```

```
    END IF;
END IF;
END;
```

▪ Exemple 2: Trigger niveau table

```
CREATE OR REPLACE TRIGGER trg_BIU_Employee
    BEFORE INSERT OR UPDATE ON Employee

BEGIN
    IF INSERTING THEN
        IF USER!= 'HRDBI1' THEN
            RAISE_APPLICATION_ERROR (-20001,'Utilisateur non autorisee.');
```

```
        END IF;
    END IF;

    IF UPDATING THEN
        DBMS_OUTPUT.PUT_LINE('Mise a jour de 1 employee.');
```

```
    END IF;

END;
```

▪ Exemple 2: Trigger niveau table (suite ...)

```
-- user HRDBI1
```

```
INSERT INTO Employee (EmployeeId,First_Name,Last_Name,Email,Hire_Date,JobId)  
VALUES (Seq_Employee.NEXTVAL,'Jo','Helisson','jhelisson@yahoo.fr',SYSDATE,'SH_CLERK');
```

```
UPDATE Employee SET Hire_Date = Hire_Date -3 WHERE EmployeeId = (SELECT  
Seq_Employee.CURRVAL FROM DUAL);
```

```
-- user SCOTT
```

```
INSERT INTO Employee (EmployeeId,First_Name,Last_Name,Email,Hire_Date,JobId)  
VALUES (208,'Alana','Fay','alanafay@yahoo.fr',SYSDATE,'SH_CLERK');
```

```
UPDATE Employee SET Hire_Date = Hire_Date -3 WHERE EmployeeId = 207;
```

▪ Exemple 1 : Trigger niveau ligne

```
CREATE OR REPLACE TRIGGER trg_AUD_Employee
AFTER UPDATE OR DELETE ON Employee
FOR EACH ROW
v1_Action VARCHAR2(20);

BEGIN
    v1_Action := 'SUPPRESSION';
    IF UPDATING THEN
        v1_Action := 'MODIFICATION';
    END IF;
    INSERT INTO Histo_Employee
    VALUES (SYSDATE, :OLD.EmployeeId, :OLD.First_Name, :OLD.Last_Name,
            :OLD.Salary, v1_Action);
END;
```



TRIGGER AFTER : à des fins de statistiques ou de journalisation

▪ Exemple 2 : Trigger niveau ligne

```
CREATE OR REPLACE TRIGGER audit_emp_values
AFTER DELETE OR INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_emp(user_name, time_stamp, id,
        old_last_name, new_last_name, old_title,
        new_title, old_salary, new_salary)
    VALUES (USER, SYSDATE, :OLD.employee_id,
        :OLD.last_name, :NEW.last_name, :OLD.job_id,
        :NEW.job_id, :OLD.salary, :NEW.salary);
END;
/
```

▪ Exemple 3 : Trigger niveau ligne

```
CREATE OR REPLACE TRIGGER trg_BI_Employee
BEFORE INSERT ON Employee
FOR EACH ROW

BEGIN
    IF (:NEW.EmployeeId IS NULL) THEN
        SELECT Seq_Employee.NEXTVAL INTO :NEW.EmployeeId FROM DUAL;
    END IF;
    SELECT USER INTO :NEW.User_Name FROM DUAL;
    :NEW.Update_Date := TO_CHAR(SYSTIMESTAMP, 'YYMMDDHH24MISS');
END;
```



*TRIGGER **BEFORE** : permet de contrôler la saisie à priori ou de pré-remplir certaines colonnes avant insertion de la ligne dans la base.*

▪ Exemple 4 : Trigger niveau ligne

```
CREATE OR REPLACE TRIGGER restrict_salary
BEFORE INSERT OR UPDATE OF salary ON employees
FOR EACH ROW
BEGIN
    IF NOT (:NEW.job_id IN ('AD_PRES', 'AD_VP'))
        AND :NEW.salary > 15000 THEN
        RAISE_APPLICATION_ERROR (-20202,
            'Employee cannot earn more than $15,000.');
```

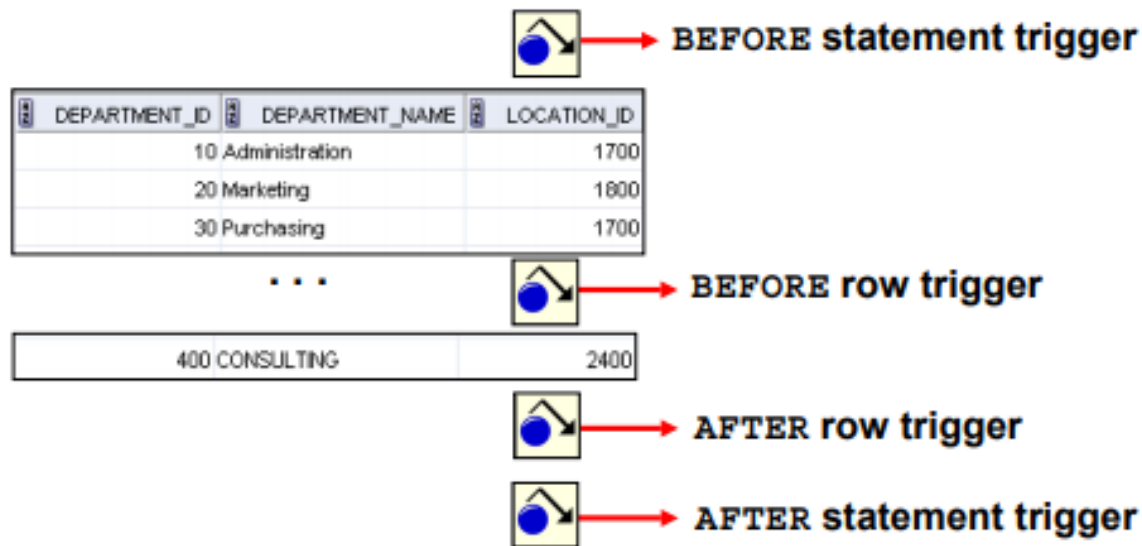
```
END IF;
END;/

UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell';
```

```
Error starting at line 1 in command:
UPDATE employees
SET salary = 15500
WHERE last_name = 'Russell'
Error report:
SQL Error: ORA-20202: Employee cannot earn more than $15,000.
ORA-06512: at "ORA62.RESTRICT_SALARY", line 4
ORA-04088: error during execution of trigger 'ORA62.RESTRICT_SALARY'
```

▪ Résumé : Trigger niveau table et niveau ligne

```
INSERT INTO departments  
  (department_id, department_name, location_id)  
VALUES (400, 'CONSULTING', 2400);
```



Les Triggers DDL



- **Quand :** BEFORE | AFTER
 - Déclenché avant ou après l'évènement DDL

- **Quoi :**
 - CREATE
 - ALTER
 - DROP
 - TRUNCATE
 - GRANT
 - RENAME ...

▪ Syntaxe :

```
CREATE [OR REPLACE] TRIGGER [schema.]nomtrigger  
    timing ON SCHEMA  
BEGIN  
    Trigger_Body  
END;
```



▪ Exemple 1 et 2 : Trigger DDL

```
CREATE OR REPLACE TRIGGER trg_ACOS
AFTER CREATE ON SCHEMA

BEGIN
    INSERT INTO Log_Table VALUES (USER, SYSDATE, 'CREATE');
END;
```

```
CREATE OR REPLACE TRIGGER trg_BDOS
BEFORE DROP ON SCHEMA

BEGIN
    INSERT INTO Log_Table VALUES (USER, SYSDATE, 'DROP');
END;
```

▪ Exemple 3 : Trigger DDL

```
CREATE OR REPLACE TRIGGER trg_BDOS  
BEFORE DROP ON SCHEMA  
  
BEGIN  
    RAISE_APPLICATION_ERROR (-20203, 'Tentative de Drop - Non Autorisee');  
END;
```

Les Triggers SYSTEM ou (database)



- **Quand :** BEFORE | AFTER
 - Déclenché avant ou après l'évènement

- **Quoi :**
 - AFTER LOGON
 - AFTER STARTUP
 - BEFORE LOGOFF
 - BEFORE SHUTDOWN
 - AFTER SERVERERROR

▪ Syntaxe :

```
CREATE [OR REPLACE] TRIGGER [schema.]nomtrigger  
    timing  
    [database_Event1 [OR [database_Event2 OR ...]]  
    ON [SCHEMA | DATABASE]  
    Trigger_Body  
END;
```



▪ Exemples 1 et 2 : Trigger SYSTEM (ou Database)

```
CREATE OR REPLACE TRIGGER trg_ALOS  
AFTER LOGON ON SCHEMA
```

```
BEGIN  
    INSERT INTO Log_Table VALUES (USER,SYSDATE,'Logging on');  
END;
```

```
CREATE OR REPLACE TRIGGER trg_BLOS  
BEFORE LOGOFF ON SCHEMA
```

```
BEGIN  
    INSERT INTO Log_Table VALUES (USER,SYSDATE,'Logging off');  
END;
```

▪ Environnement Professionnel :



Ne pas confondre l'AUDIT Oracle et l'utilisation des Triggers System ou Database

Les Triggers **INSTEAD OF**



- **Quand** : BEFORE | AFTER
 - Permettent d'arrêter et de rediriger les effets d'une instruction DML

Les Triggers **COMPOSES (COMPOUND TRIGGER)**

- **Quand** : BEFORE | AFTER
 - Permettent de combiner 4 déclencheurs dans un seul :
 - BEFORE STATEMENT et AFTER STATEMENT
 - BEFORE ROW et AFTER ROW

Les Triggers



▪ Droits

```
GRANT CREATE TRIGGER TO <user>|<role>;  
REVOKE CREATE TRIGGER FROM <user>|<role>;
```

▪ Suppression :

```
DROP TRIGGER Trigger_name;
```

▪ Recompilation

```
ALTER TRIGGER Trigger_name COMPILE;
```

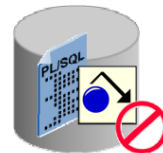
▪ Modification

Recréer le trigger avec l'option : CREATE [OR REPLACE] TRIGGER ...

Les Triggers

▪ Désactiver / Réactiver

```
ALTER TRIGGER Trigger_Name DISABLE;  
ALTER TRIGGER Trigger_Name ENABLE;
```



▪ Vues du dictionnaire de données :

```
USER_TRIGGERS  
ALL_TRIGGERS  
DBA_TRIGGERS
```

Les Triggers : en résumé



▪ Environnement Professionnel :



- mettre en oeuvre règles métiers (appel procedures stockées)
- effectuer des controles a priori (avant implementation dans la base)
- historiser des données avant ou après leur modification (dénormalisation, standby database)

Pour aller plus loin ...

Oracle Database 11g – Develop PL/SQL Program Units – Vol. 1
756 pages – (Pdf) – 4 314 Ko



Agenda

- ☒ Les séquences
- ☒ Les triggers



Questions ?



Fin !

GET READY FOR THE FUTURE

